

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 2

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement :

Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1. Import the required libraries.
2. Load the Wine dataset from sklearn.datasets.
3. Separate the dataset into features (X) and target labels (y).
4. Split the dataset into training and testing sets.
5. Standardize the feature values using StandardScaler.
6. Create a Multilayer Perceptron (MLP) classifier with:
 - Two hidden layers of 10 neurons each
 - ReLU activation function
 - Adam optimizer
 - Maximum 1000 iterations
7. Train the MLP model using the training data.
8. Predict the class labels for the test data.
9. Evaluate the model using:
 - Accuracy Score
 - Confusion Matrix
 - Classification Report
10. Visualize the results using:

- Feature Correlation Heatmap
- Confusion Matrix Heatmap

11. Analyze the model's performance and draw conclusions.

Code & Output :

```
3]: from sklearn.datasets import load_wine
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.neural_network import MLPClassifier
    from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

4]: wine=load_wine()
    X,y = wine.data, wine.target

5]: X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)

6]: scaler=StandardScaler()
    X_train=scaler.fit_transform(X_train)
    X_test=scaler.transform(X_test)

7]: mlp=MLPClassifier(hidden_layer_sizes=(10,10),activation='relu',solver='adam',max_iter=1000,random_state=42)
    mlp.fit(X_train,y_train)
    y_pred=mlp.predict(X_test)
    print("Accuracy:",accuracy_score(y_test,y_pred))

    print("Confusion Matrix:\n",confusion_matrix(y_test,y_pred))
    print("\nClassification Report:\n",classification_report(y_test,y_pred))
```

Accuracy: 0.9814814814814815

Confusion Matrix:

```
[[19  0  0]
 [ 0 20  1]
 [ 0  0 14]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	0.95	0.98	21
2	0.93	1.00	0.97	14
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54

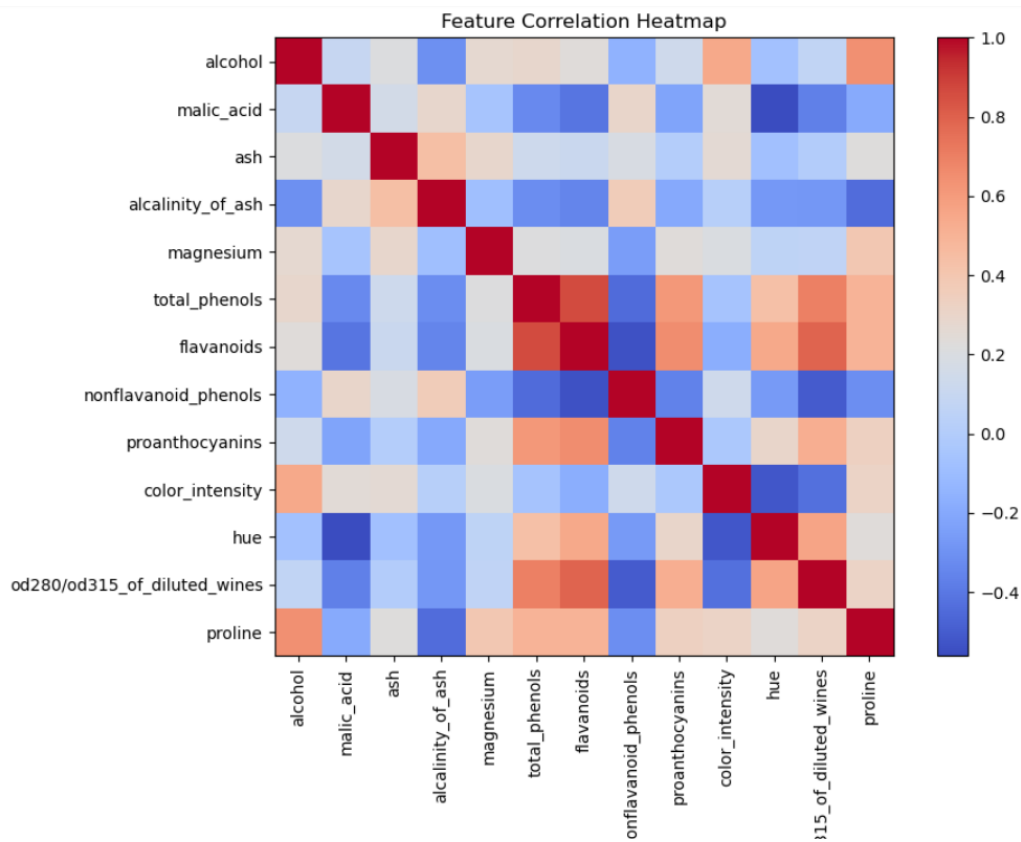
```
[8]: import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame(wine.data, columns=wine.feature_names)

plt.figure(figsize=(10,8))
plt.imshow(df.corr(), cmap='coolwarm', interpolation='nearest')
plt.colorbar()

plt.xticks(range(len(df.columns)), df.columns, rotation=90)
plt.yticks(range(len(df.columns)), df.columns)

plt.title("Feature Correlation Heatmap")
plt.tight_layout()
plt.show()
```



```

: cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
plt.imshow(cm, cmap='Blues')

plt.title("Confusion Matrix")
plt.colorbar()

tick_marks = range(len(wine.target_names))
plt.xticks(tick_marks, wine.target_names)
plt.yticks(tick_marks, wine.target_names)

for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j],
                 ha="center", va="center",
                 color="white" if cm[i, j] > cm.max()/2 else "black")

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.tight_layout()
plt.show()

```

